

Joins

```
CREATE DATABASE J;
USE J;
CREATE TABLE Department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(30)
);
CREATE TABLE Student (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(30),
    dept_id INT
);
CREATE TABLE StudentMentor (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(30),
    mentor_id INT
);
INSERT INTO Department VALUES
(10, 'Computer Science'),
(20, 'Information Science'),
(30, 'Data Science'),
(40, 'Electronics');
INSERT INTO Student VALUES
(1, 'Manu', 10),
(2, 'Varun', 20),
(3, 'Vaibhav', 30),
(4, 'Raghavendra', NULL);
INSERT INTO StudentMentor VALUES
(1, 'Manu', NULL),
(2, 'Varun', 1),
(3, 'Vaibhav', 1),
(4, 'Raghavendra', 2);
```

1. Inner Join: Students enrolled in existing departments

```
SELECT s.student_id, s.student_name, d.dept_name
FROM Student s
INNER JOIN Department d ON s.dept_id = d.dept_id
;
```

Output :

Result Grid				Filter Rows:		Export:		Wrap Cell Content:	IA
	student_id	student_name	dept_name						
▶	1	Manu	Computer Science						
	2	Varun	Information Science						
	3	Vaibhav	Data Science						

2. Left Join

```
SELECT s.student_id, s.student_name, d.dept_name
FROM Student s
LEFT JOIN Department d ON s.dept_id = d.dept_id;
```

Result Grid				Filter Rows:		Export:
	student_id	student_name	dept_name			
▶	1	Manu	Computer Science			
	2	Varun	Information Science			
	3	Vaibhav	Data Science			
	4	Raghavendra	NULL			

3. Right Join

```
SELECT s.student_id, s.student_name, d.dept_name
FROM Student s
RIGHT JOIN Department d ON s.dept_id = d.dept_id;
```

Result Grid

Filter Rows:

Export:

Wrap Cell Content:

	student_id	student_name	dept_name
▶	1	Manu	Computer Science
	2	Varun	Information Science
	3	Vaibhav	Data Science
	NULL	NULL	Electronics

4. Fully Outer Join

```

SELECT s.student_id, s.student_name, d.dept_name
FROM Student s
LEFT JOIN Department d ON s.dept_id = d.dept_id
UNION SELECT s.student_id, s.student_name, d.dept_name
FROM Student s
RIGHT JOIN Department d ON s.dept_id = d.dept_id;

```

Result Grid

 Filter Rows:

Export: 

	student_id	student_name	dept_name
▶	1	Manu	Computer Science
	2	Varun	Information Science
	3	Vaibhav	Data Science
	4	Raghavendra	NULL
	NULL	NULL	Electronics

5. Cross Join

```

SELECT s.student_name, d.dept_name
FROM Student s
CROSS JOIN Department d;

```

Result Grid			Filter Rows:	Export:
	student_name	dept_name		
▶	Raghavendra	Computer Science		
	Vaibhav	Computer Science		
	Varun	Computer Science		
	Manu	Computer Science		
	Raghavendra	Information Science		
	Vaibhav	Information Science		
	Varun	Information Science		
	Manu	Information Science		
	Raghavendra	Data Science		
	Vaibhav	Data Science		
	Varun	Data Science		
	Manu	Data Science		
	Raghavendra	Electronics		
	Vaibhav	Electronics		
	Varun	Electronics		
	Manu	Electronics		

6. Self Join

```
SELECT s.student_name AS student, m.student_name AS mentor
FROM StudentMentor s
INNER JOIN StudentMentor m ON s.mentor_id = m.student_id;
```

Result Grid			Filter Rows:
	student	mentor	
▶	Varun	Manu	
	Vaibhav	Manu	
	Raghavendra	Varun	

LAB 8

Group by having order by clause

Group By

```
CREATE DATABASE P;
USE P;
CREATE TABLE Student (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(30),
    dept VARCHAR(30),
    marks INT
);
INSERT INTO Student VALUES
(1, 'Manu', 'CS', 85),
(2, 'Vaibhav', 'IS', 72),
(3, 'Varun', 'DS', 90),
(4, 'Ragha', 'DS', 78),
(5, 'Priya', 'CS', 88),
(6, 'Kiran', 'IS', 65),
(7, 'Sneha', 'DS', 95);
SELECT student_name, dept, marks
FROM Student
ORDER BY marks DESC;
```

Output :

Result Grid  Filter Rows:			
	student_name	dept	marks
▶	Sneha	DS	95
	Varun	DS	90
	Priya	CS	88
	Manu	CS	85
	Ragha	DS	78
	Vaibhav	IS	72
	Kiran	IS	65

Having By :

```
SELECT dept, COUNT(*) AS total_students
FROM Student
GROUP BY dept
HAVING COUNT(*) > 2;
```

Result Grid  Filter Rows:		
	dept	total_students
▶	DS	3

Order By :

```
SELECT dept,
COUNT(*) AS total_students,
SUM(marks) AS total_marks,
AVG(marks) AS avg_marks
FROM Student
GROUP BY dept;
```

Result Grid					Filter Rows:	Export:
	dept	total_students	total_marks	avg_marks		
▶	CS	2	173	86.5000		
	IS	2	137	68.5000		
	DS	3	263	87.6667		

LAB 9

Procedure

Basic Procedure :

```

CREATE DATABASE S;
USE S;
CREATE TABLE Student (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(30),
    dept VARCHAR(30),
    marks INT
);
INSERT INTO Student VALUES
(1, 'Manu', 'CS', 85),
(2, 'Vaibhav', 'IS', 72),
(3, 'Varun', 'DS', 90),
(4, 'Ragha', 'DS', 78),
(5, 'Priya', 'CS', 88),
(6, 'Kiran', 'IS', 65),
(7, 'Sneha', 'DS', 95);
DROP PROCEDURE IF EXISTS show_students;
DELIMITER $$
CREATE PROCEDURE show_students()
BEGIN
    SELECT * FROM Student;

```

```
END$$
DELIMITER ;
CALL show_students();
```

Output :

Result Grid  Filter Rows: Export:  Wra				
	student_id	student_name	dept	marks
▶	1	Manu	CS	85
	2	Vaibhav	IS	72
	3	Varun	DS	90
	4	Ragha	DS	78
	5	Priya	CS	88
	6	Kiran	IS	65
	7	Sneha	DS	95

Procedure with in parameter :

```
DELIMITER $$
CREATE PROCEDURE insert_student(
    IN p_id INT,
    IN p_name VARCHAR(30),
    IN p_dept VARCHAR(30),
    IN p_marks INT
)
BEGIN
    INSERT INTO Student VALUES (p_id, p_name, p_dept, p_marks);
END$$
DELIMITER ;
CALL insert_student(8, 'Arun', 'IS', 80);
SELECT * FROM Student;
```

Result Grid   Filter Rows:				
	student_id	student_name	dept	marks
▶	1	Manu	CS	85
	2	Vaibhav	IS	72
	3	Varun	DS	90
	4	Ragha	DS	78
	5	Priya	CS	88
	6	Kiran	IS	65
	7	Sneha	DS	95
	8	Arun	IS	80
●	NULL	NULL	NULL	NULL

Procedure for update :

```

DELIMITER $$
CREATE PROCEDURE update_marks(
    IN p_id INT,
    IN p_new_marks INT
)
BEGIN
    UPDATE Student
    SET marks = p_new_marks
    WHERE student_id = p_id;
END$$
DELIMITER ;

```

```

CALL update_marks(2, 79);
SELECT * FROM Student;

```

Result Grid				
Filter Rows:				
	student_id	student_name	dept	marks
▶	1	Manu	CS	85
	2	Vaibhav	IS	79
	3	Varun	DS	90
	4	Ragha	DS	78
	5	Priya	CS	88
	6	Kiran	IS	65
	7	Sneha	DS	95
✱	NULL	NULL	NULL	NULL

Procedure for delete :

```

DELIMITER $$
CREATE PROCEDURE insert_student(
    IN p_id INT,
    IN p_name VARCHAR(30),
    IN p_dept VARCHAR(30),
    IN p_marks INT
)
DELIMITER $$
CREATE PROCEDURE delete_student(
    IN p_id INT
)
BEGIN
    DELETE FROM Student
    WHERE student_id = p_id;
END$$
DELIMITER ;
CALL delete_student(5);
SELECT * FROM Student;

```

Result Grid  Filter Rows:				
	student_id	student_name	dept	marks
▶	1	Manu	CS	85
	2	Vaibhav	IS	72
	3	Varun	DS	90
	4	Ragha	DS	78
	6	Kiran	IS	65
	7	Sneha	DS	95
✱	NULL	NULL	NULL	NULL

10. Triggers

-- Table Setup

```
CREATE TABLE Student (
    student_id INT PRIMARY KEY,
    student_name VARCHAR(30),
    dept VARCHAR(30),
    marks INT
);
```

```
CREATE TABLE Student_log (
    log_id INT AUTO_INCREMENT PRIMARY KEY,
    action VARCHAR(30),
    student_id INT,
    log_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

INSERT INTO Student VALUES

```
(1, 'Asha', 'CS', 85),
(2, 'Ravi', 'IS', 72),
(3, 'Meena', 'DS', 90),
(4, 'John', 'DS', 78),
(5, 'Priya', 'CS', 88),
(6, 'Kiran', 'IS', 65),
(7, 'Sneha', 'DS', 95);
```

```
-- Drop triggers if they exist
DROP TRIGGER IF EXISTS before_student_insert;
DROP TRIGGER IF EXISTS after_student_insert;
DROP TRIGGER IF EXISTS before_student_update;
DROP TRIGGER IF EXISTS after_student_update;
DROP TRIGGER IF EXISTS before_student_delete;
DROP TRIGGER IF EXISTS after_student_delete;
```

```
DELIMITER $$
```

```
-- 1. BEFORE INSERT: Validate marks (marks cannot be negative or over 100)
```

```
CREATE TRIGGER before_student_insert
BEFORE INSERT ON Student
FOR EACH ROW
BEGIN
    IF NEW.marks < 0 OR NEW.marks > 100 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Marks must be between 0 and 100';
    END IF;
END$$
```

```
-- 2. AFTER INSERT: Log insert operation
```

```
CREATE TRIGGER after_student_insert
AFTER INSERT ON Student
FOR EACH ROW
BEGIN
    INSERT INTO Student_log (action, student_id) VALUES ('INSERT', NEW.student_id);
END$$
```

```
-- 3. BEFORE UPDATE: Prevent reduction of marks
```

```
CREATE TRIGGER before_student_update
BEFORE UPDATE ON Student
FOR EACH ROW
BEGIN
    IF NEW.marks < OLD.marks THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Marks cannot be reduced';
    END IF;
END$$
```

```
-- 4. AFTER UPDATE: Log update operation
```

```
CREATE TRIGGER after_student_update
AFTER UPDATE ON Student
FOR EACH ROW
```

```

BEGIN
    INSERT INTO Student_log (action, student_id) VALUES ('UPDATE', NEW.student_id);
END$$

-- 5. BEFORE DELETE: Prevent deleting a distinction student (marks > 90)
CREATE TRIGGER before_student_delete
BEFORE DELETE ON Student
FOR EACH ROW
BEGIN
    IF OLD.marks > 90 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cannot delete student with marks above 90';
    END IF;
END$$

-- 6. AFTER DELETE: Log delete operation
CREATE TRIGGER after_student_delete
AFTER DELETE ON Student
FOR EACH ROW
BEGIN
    INSERT INTO Student_log (action, student_id) VALUES ('DELETE', OLD.student_id);
END$$

DELIMITER ;

-- Verification / Test Queries
INSERT INTO Student VALUES (8, 'Arun', 'IS', 80);
INSERT INTO Student VALUES (9, 'Divya', 'DS', 89);

UPDATE Student SET marks = 75 WHERE student_id = 2;
UPDATE Student SET marks = 92 WHERE student_id = 1;

DELETE FROM Student WHERE student_id = 6;
DELETE FROM Student WHERE student_id = 5;

SELECT * FROM Student;
SELECT * FROM Student_log;

```

Output :

Result Grid



Filter Rows:

Edit:



	log_id	action	student_id	log_time
▶	1	INSERT	8	2026-10-09 21:49:53
	2	INSERT	9	2026-10-09 21:49:53
	3	UPDATE	2	2026-10-09 21:49:53
	4	UPDATE	1	2026-10-09 21:49:53
	5	DELETE	6	2026-10-09 21:49:53
	6	DELETE	5	2026-10-09 21:49:53
•	NULL	NULL	NULL	NULL